

Real-Time Examples of the Prototype Design Pattern in C#

In this article, I will discuss the **Real-Time Examples of the Prototype Design Pattern in C#**. Please read our previous article discussing the basic concepts of the [Prototype Design Pattern in C#](#) with Examples. At the end of this article, you will understand the following Real-time Examples using the Prototype Design Pattern in C#.

1. [Car Configurator Application](#)
2. [Design Firm that deals with Floor Plans of Buildings](#)
3. [Document Versioning System](#)
4. [Software Application for an Event Management Company](#)
5. [Building a User-Customizable GUI System](#)
6. [Graphics Editors](#)
7. [Database Record Duplication](#)
8. [Game Development](#)
9. [Configuration Templates](#)
10. [Historical States for Undo Actions](#)

Real-Time Example of Prototype Design Pattern in C#: Car Configurator Application

Consider a scenario where you're working on a Car Configurator application. Users can choose and customize a car's attributes, and once they've configured a car, they might want to create another similar configuration with a few differences. The Prototype pattern is a perfect fit for this. Let us see how we can implement the above example using the Prototype Design Pattern in C#:

```
using System;
namespace PrototypeDesignPattern
{
    //Prototype - ICarPrototype Interface
    public interface ICarPrototype
    {
        ICarPrototype Clone();
    }
    //Concrete Prototype - Car Class
    public class Car : ICarPrototype
    {
        public string Model { get; set; }
        public string Color { get; set; }
        public string Engine { get; set; }
        public bool Sunroof { get; set; }
        public ICarPrototype Clone()
        {
            // Using MemberwiseClone for simplicity, which is a shallow
            copy.
            // For complex objects, you might need to implement a deep
            copy.
            return (ICarPrototype)this.MemberwiseClone();
        }
        public override string ToString()
        {
            return $"{Model} | Color: {Color} | Engine: {Engine} | Sunroof:
{Sunroof}";
        }
    }
    //Client Code
```

```
//Testing the Prototype Design Pattern
public class Program
{
    public static void Main()
    {
        // Create an initial car configuration
        Car prototypeCar = new Car
        {
            Model = "Sedan",
            Color = "Blue",
            Engine = "V6",
            Sunroof = true
        };
        Console.WriteLine("Original Car Configuration:");
        Console.WriteLine(prototypeCar);
        // Now clone the prototype and make modifications for a new
configuration
        Car clonedCar = (Car)prototypeCar.Clone();
        clonedCar.Color = "Red";
        clonedCar.Sunroof = false;
        Console.WriteLine("\nCloned and Modified Car Configuration:");
        Console.WriteLine(clonedCar);
        Console.ReadKey();
    }
}
}
```

When you run the above code, you'll see that the clonedCar has the same attributes as the prototypeCar but with a different color and without a sunroof. Using the Prototype pattern, we can easily create new car configurations by cloning and modifying existing ones. When you run the above code, you will get the following output.

```
Original Car Configuration:
Sedan | Color: Blue | Engine: V6 | Sunroof: True

Cloned and Modified Car Configuration:
Sedan | Color: Red | Engine: V6 | Sunroof: False
```

Real-Time Example of Prototype Design Pattern in C#: Design Firm that deals with Floor Plans of Buildings

Suppose you are developing a software application for a design firm that deals with floor plans of buildings. The design firm frequently uses standard room templates but occasionally tweaks them for specific clients. Quickly clone a standard room and modify it if necessary. Let us see how we can implement the above example using the Prototype Design Pattern in C#:

```
using System;
namespace PrototypeDesignPattern
{
    //Prototype - IRoomPrototype Interface
    public interface IRoomPrototype
    {
        IRoomPrototype Clone();
    }
}
```

```

//Concrete Prototype - Room Class
public class Room : IRoomPrototype
{
    public int NumberOfDoors { get; set; }
    public int NumberOfWindows { get; set; }
    public string FloorType { get; set; }
    public string WallColor { get; set; }
    public IRoomPrototype Clone()
    {
        // Using MemberwiseClone for simplicity, which is a shallow
copy.
        return (IRoomPrototype)this.MemberwiseClone();
    }
    public override string ToString()
    {
        return $"Room with {NumberOfDoors} doors, {NumberOfWindows}
windows, {FloorType} floor, and {WallColor} walls.";
    }
}

//Client Code
//Testing the Prototype Design Pattern
public class Program
{
    public static void Main()
    {
        // Standard room template
        Room standardRoom = new Room
        {
            NumberOfDoors = 1,
            NumberOfWindows = 2,
            FloorType = "Hardwood",
            WallColor = "White"
        };
        Console.WriteLine("Standard Room:");
        Console.WriteLine(standardRoom);
        // Client wants a similar room but with tiles and an extra
window.
        Room customRoom = (Room)standardRoom.Clone();
        customRoom.FloorType = "Tiles";
        customRoom.NumberOfWindows = 3;
        Console.WriteLine("\nCustomized Room:");
        Console.WriteLine(customRoom);
        Console.ReadKey();
    }
}

```

When executed, the program first displays the standard room configuration. Next, it demonstrates the Prototype pattern's advantage by quickly creating a customized room based on the standard, followed by a few modifications.

This example showcases how the Prototype pattern can be valuable when creating a new instance of an object from scratch is more cumbersome or resource-intensive than duplicating an existing one and then making a few adjustments. When you run the above code, you will get the following output.

Standard Room:

Room with 1 doors, 2 windows, Hardwood floor, and White walls.

Customized Room:

Room with 1 doors, 3 windows, Tiles floor, and White walls.

Real-Time Example of Prototype Design Pattern in C#: Document Versioning System

Suppose we're developing a "Document Versioning System" where users can save versions of their documents without consuming a lot of storage. Instead of storing each version as a separate entity, we might only store the changes (deltas) but still need the ability to reproduce any document version instantly. Let us see how we can implement the above example using the Prototype Design Pattern in C#:

```
using System;
namespace PrototypeDesignPattern
{
    //Prototype - DocumentPrototype Interface
    public interface IDocumentPrototype
    {
        IDocumentPrototype Clone();
    }
    //Concrete Prototype - Document Class
    public class Document : IDocumentPrototype
    {
        public string Title { get; set; }
        public string Content { get; set; }
        public IDocumentPrototype Clone()
        {
            return new Document
            {
                Title = this.Title,
                Content = this.Content
            };
        }
        public void Display()
        {
            Console.WriteLine($"Title: {Title}\nContent: {Content}\n");
        }
    }

    //Client Code
    //Testing the Prototype Design Pattern
    public class Program
    {
        public static void Main()
        {
            Document originalDoc = new Document
            {
                Title = "Prototype Design Pattern",
                Content = "This is a document explaining the prototype
design pattern."
            };
            Console.WriteLine("Original Document:");
```

```

        originalDoc.Display();
        // The user now edits the document.
        Document versionedDoc = (Document)originalDoc.Clone();
        versionedDoc.Content += "\nNow, the content has been updated to
include more details.";
        Console.WriteLine("Versioned Document:");
        versionedDoc.Display();
        // The original document remains unchanged, but we have a new
version saved.
        // In reality, we might just save the delta (changes) for
efficiency.
        Console.ReadKey();
    }
}
}

```

Here, whenever a user decides to save a new version of their document, the system clones the current document and applies the changes. The user can revert to any previous version, and the system would rebuild that version using the original prototype and applying saved deltas in sequence.

This example illustrates how the Prototype pattern can be handy in scenarios where we need to quickly duplicate an object and possibly modify it without affecting the original. When you run the above code, you will get the following output.

```

Original Document:
Title: Prototype Design Pattern
Content: This is a document explaining the prototype design pattern.

Versioned Document:
Title: Prototype Design Pattern
Content: This is a document explaining the prototype design pattern.
Now, the content has been updated to include more details.

```

Real-Time Example of Prototype Design Pattern in C#: Software Application for an Event Management Company

Suppose you're developing a software application for an event management company that organizes conferences. When organizing a conference, the company follows a standard template of a schedule. However, minor tweaks or customizations might be needed for each client based on client preferences, speakers available, etc.

Clone the standard conference schedule and adjust it as required for different clients without recreating the entire schedule from scratch. Let us see how we can implement the above example using the Prototype Design Pattern in C#:

```

using System;
using System.Collections.Generic;
namespace PrototypeDesignPattern
{
    //Prototype - ISchedulePrototype Interface
    public interface ISchedulePrototype
    {
        ISchedulePrototype Clone();
    }
    //Concrete Prototype - ConferenceSchedule Class

```

```

public class ConferenceSchedule : ISchedulePrototype
{
    public List<string> Sessions { get; set; } = new List<string>();
    public List<string> Speakers { get; set; } = new List<string>();
    public ISchedulePrototype Clone()
    {
        // Deep copying is important here since Lists are reference
types.
        return new ConferenceSchedule
        {
            Sessions = new List<string>(this.Sessions),
            Speakers = new List<string>(this.Speakers)
        };
    }
    public void DisplaySchedule()
    {
        Console.WriteLine("Conference Schedule:");
        for (int i = 0; i < Sessions.Count; i++)
        {
            Console.WriteLine($"Session {i + 1}: {Sessions[i]} by
{Speakers[i]}");
        }
        Console.WriteLine();
    }
}

//Client Code
//Testing the Prototype Design Pattern
public class Program
{
    public static void Main()
    {
        // Standard conference schedule template
        ConferenceSchedule standardSchedule = new ConferenceSchedule
        {
            Sessions = { "Welcome Note", "Tech Trends", "Networking
Session" },
            Speakers = { "Host", "Tech Guru", "All Participants" }
        };
        Console.WriteLine("Standard Schedule:");
        standardSchedule.DisplaySchedule();
        // A client wants a similar schedule but with an added
"Innovation Talk".
        ConferenceSchedule customSchedule =
(ConferenceSchedule)standardSchedule.Clone();
        customSchedule.Sessions.Insert(2, "Innovation Talk");
        customSchedule.Speakers.Insert(2, "Innovation Expert");
        Console.WriteLine("Custom Schedule for Client A:");
        customSchedule.DisplaySchedule();
        Console.ReadKey();
    }
}
}

```

This example illustrates how the Prototype pattern can be beneficial in scenarios where an existing template or structure needs to be replicated and customized for different requirements, reducing the overhead of creating everything from scratch and ensuring consistency across variants. When you run the above code, you will get the following output.

```
Standard Schedule:
Conference Schedule:
Session 1: Welcome Note by Host
Session 2: Tech Trends by Tech Guru
Session 3: Networking Session by All Participants

Custom Schedule for Client A:
Conference Schedule:
Session 1: Welcome Note by Host
Session 2: Tech Trends by Tech Guru
Session 3: Innovation Talk by Innovation Expert
Session 4: Networking Session by All Participants
```

Real-Time Example of Prototype Design Pattern in C#: Building a User-Customizable GUI System

Imagine you're building a user-customizable GUI system. Users can create various styled buttons with intricate designs, and they want to reuse them to create similarly styled buttons but with different labels. Users should be able to clone a styled button and then change its label without affecting the original button. Let us see how we can implement the above example using the Prototype Design Pattern in C#:

```
using System;
namespace PrototypeDesignPattern
{
    //Prototype - IButtonPrototype Interface
    public interface IButtonPrototype
    {
        IButtonPrototype Clone();
    }
    //Concrete Prototype - StyledButton Class
    public class StyledButton : IButtonPrototype
    {
        public string Label { get; set; }
        public string Color { get; set; }
        public int Width { get; set; }
        public int Height { get; set; }
        public IButtonPrototype Clone()
        {
            return new StyledButton
            {
                // Copy the style attributes
                Color = this.Color,
                Width = this.Width,
                Height = this.Height,
                // New label can be set by the user
                Label = ""
            };
        }
    }
}
```

```

        };
    }
    public void Display()
    {
        Console.WriteLine($"Button: {Label} | Color: {Color} | Size:
{Width}x{Height}");
    }
}

//Client Code
//Testing the Prototype Design Pattern
public class Program
{
    public static void Main()
    {
        // User creates a styled button
        StyledButton originalButton = new StyledButton
        {
            Label = "Submit",
            Color = "Blue",
            Width = 100,
            Height = 50
        };
        Console.WriteLine("Original Button:");
        originalButton.Display();
        // User wants a new button with the same style but a different
label
        StyledButton clonedButton =
(StyledButton)originalButton.Clone();
        clonedButton.Label = "Cancel";
        Console.WriteLine("Cloned Button:");
        clonedButton.Display();
        Console.ReadKey();
    }
}
}

```

Upon executing this, you'll have an original button with the label "Submit" and a cloned button with the label "Cancel". Both buttons share the same style properties, but cloning allowed the label of the second button to be set independently of the first.

A real GUI application could add more attributes (like font, border style, etc.). Still, the principle remains: use the Prototype pattern to easily and efficiently duplicate and customize objects based on a prototype. When you run the above code, you will get the following output.

```

Original Button:
Button: Submit | Color: Blue | Size: 100x50
Cloned Button:
Button: Cancel | Color: Blue | Size: 100x50

```

Real-Time Example of Prototype Design Pattern in C#: Graphics Editors

Graphics editors often involve objects (like shapes) that users may want to replicate with slight modifications, making the Prototype pattern a natural fit. Here's how this could be implemented in a simple graphics editor

scenario:

- **Scenario:** Imagine a graphics editor where users can draw shapes and replicate (clone) these shapes to create patterns or designs.
- **Objective:** Allow users to clone a drawn shape and modify its properties (like position or color) without altering the original shape.

Let us see how we can implement the above example using the Prototype Design Pattern in C#:

```
using System;
namespace PrototypeDesignPattern
{
    //Prototype - IShapePrototype Interface
    public interface IShapePrototype
    {
        IShapePrototype Clone();
    }
    //Concrete Prototype - Circle Class
    public class Circle : IShapePrototype
    {
        public int X { get; set; } // X-coordinate of the circle's center
        public int Y { get; set; } // Y-coordinate of the circle's center
        public int Radius { get; set; }
        public string Color { get; set; }
        public IShapePrototype Clone()
        {
            return new Circle
            {
                X = this.X,
                Y = this.Y,
                Radius = this.Radius,
                Color = this.Color
            };
        }
        public void Display()
        {
            Console.WriteLine($"Circle at ({X}, {Y}) | Radius: {Radius} | Color: {Color}");
        }
    }

    //Client Code
    //Testing the Prototype Design Pattern
    public class Program
    {
        public static void Main()
        {
            // User creates a circle in the graphics editor
            Circle originalCircle = new Circle
            {
                X = 5,
                Y = 5,
                Radius = 10,
                Color = "Red"
            };
        }
    }
}
```

```

        Console.WriteLine("Original Circle:");
        originalCircle.Display();
        // The user decides to clone the circle and place it at a
different location
        Circle clonedCircle = (Circle)originalCircle.Clone();
        clonedCircle.X = 20;
        clonedCircle.Y = 20;
        Console.WriteLine("Cloned Circle with Different Position:");
        clonedCircle.Display();
        Console.ReadKey();
    }
}
}

```

In this example, the user designs a circle in the graphics editor and then decides to clone it. The Prototype pattern enables the efficient creation of a new circle that retains the original circle's properties. Then, it's modified to be placed at a different location on the canvas.

While this is a basic example, in real-world graphics editors, the prototype pattern would extend to accommodate a variety of shapes, effects, layers, and other graphical elements, providing a flexible approach to replicating and customizing designs. When you run the above code, you will get the following output.

```

Original Circle:
Circle at (5, 5) | Radius: 10 | Color: Red
Cloned Circle with Different Position:
Circle at (20, 20) | Radius: 10 | Color: Red

```

Real-Time Example of Prototype Design Pattern in C#: Database Record Duplication

Let's understand the scenario of database record duplication using the Prototype Design Pattern in C#.

Suppose you're working on an HR application where employees have profiles. Occasionally, temporary or contract staff are brought in, and their details are similar to existing employees (maybe they're from the same agency or have similar roles). Instead of manually inputting all the details again, HR wants to duplicate an existing profile and modify the necessary fields.

Efficiently clone an existing employee profile and adjust it for a new employee. Let us see how we can implement the above example using the Prototype Design Pattern in C#:

```

using System;
namespace PrototypeDesignPattern
{
    //Prototype - IEmployeePrototype Interface
    public interface IEmployeePrototype
    {
        IEmployeePrototype Clone();
    }
    //Concrete Prototype - EmployeeProfile Class
    public class EmployeeProfile : IEmployeePrototype
    {
        public string Name { get; set; }
        public string Address { get; set; }
        public string Role { get; set; }
        public decimal Salary { get; set; }
    }
}

```

```

    public IEmployeePrototype Clone()
    {
        return new EmployeeProfile
        {
            Name = this.Name,
            Address = this.Address,
            Role = this.Role,
            Salary = this.Salary
        };
    }
    public void DisplayProfile()
    {
        Console.WriteLine($"Name: {Name} | Address: {Address} | Role:
{Role} | Salary: ${Salary}");
    }
}

//Client Code
//Testing the Prototype Design Pattern
public class Program
{
    public static void Main()
    {
        // Existing employee profile in the HR database
        EmployeeProfile existingEmployee = new EmployeeProfile
        {
            Name = "John Doe",
            Address = "123 Main St, Cityville",
            Role = "Software Developer",
            Salary = 75000
        };
        Console.WriteLine("Existing Employee Profile:");
        existingEmployee.DisplayProfile();
        // HR wants to create a profile for a new temporary staff
member with a similar role and salary
        EmployeeProfile tempEmployee =
(EmployeeProfile)existingEmployee.Clone();
        tempEmployee.Name = "Jane Smith";
        tempEmployee.Address = "456 Elm St, Cityville";
        Console.WriteLine("\nNew Temporary Employee Profile:");
        tempEmployee.DisplayProfile();
        Console.ReadKey();
    }
}
}

```

In this example, an existing employee profile is cloned, and then only the fields that differ (in this case, Name and Address) are modified for the new profile. This allows the HR department to quickly and efficiently create profiles for temporary staff or others with similar attributes without manually setting each property. The Prototype pattern ensures a consistent base for the new profiles, reducing potential errors and data entry time. When you run the above code, you will get the following output.

Existing Employee Profile:

Name: John Doe | Address: 123 Main St, Cityville | Role: Software Developer | Salary: \$75000

New Temporary Employee Profile:

Name: Jane Smith | Address: 456 Elm St, Cityville | Role: Software Developer | Salary: \$75000

Real-Time Example of Prototype Design Pattern in C#: Game Development

The Prototype design pattern is frequently used in game development to instantiate multiple instances of game entities without incurring the overhead of constructing each one from scratch.

- **Scenario:** Consider a game where players can collect and deploy creatures into battles. Each creature has a base type, but they might have variations like random bonuses or color tweaks when deployed in battle. Instead of recreating every creature from scratch, the game uses the Prototype pattern to clone a base creature and apply any variations quickly.
- **Objective:** Efficiently generate battle-ready creatures by cloning a prototype and adjusting properties if needed.

Let us see how we can implement the above example using the Prototype Design Pattern in C#:

```
using System;
namespace PrototypeDesignPattern
{
    //Prototype - ICreaturePrototype Interface
    public interface ICreaturePrototype
    {
        ICreaturePrototype Clone();
    }
    //Concrete Prototype - Creature Class
    public class Creature : ICreaturePrototype
    {
        public string Type { get; set; }
        public int BaseHealth { get; set; }
        public int BaseAttack { get; set; }
        public string Color { get; set; }
        public ICreaturePrototype Clone()
        {
            return new Creature
            {
                Type = this.Type,
                BaseHealth = this.BaseHealth,
                BaseAttack = this.BaseAttack,
                Color = this.Color
            };
        }
        public void DisplayStats()
        {
            Console.WriteLine($"{Type} | Health: {BaseHealth} | Attack: {BaseAttack} | Color: {Color}");
        }
    }

    //Client Code
    //Testing the Prototype Design Pattern
    public class Program
    {
```

```

public static void Main()
{
    // Base creature prototype loaded from the game database
    Creature dragonPrototype = new Creature
    {
        Type = "Dragon",
        BaseHealth = 100,
        BaseAttack = 50,
        Color = "Red"
    };
    Console.WriteLine("Dragon Prototype:");
    dragonPrototype.DisplayStats();
    // Player deploys a dragon in battle, but there's a 10% health
bonus in this battle
    Creature battleDragon = (Creature)dragonPrototype.Clone();
    battleDragon.BaseHealth = (int)(battleDragon.BaseHealth * 1.1);
// 10% bonus
    Console.WriteLine("\nBattle Dragon with Health Bonus:");
    battleDragon.DisplayStats();
    Console.ReadKey();
}
}
}

```

In this example, when the player deploys a dragon for battle, the game clones the dragon prototype instead of creating a new dragon from scratch and then adjusts the stats based on the battle conditions (like a 10% health bonus). The Prototype pattern ensures that the game can quickly generate creatures with consistent base attributes, which is especially beneficial in fast-paced or real-time games where performance is crucial. When you run the above code, you will get the following output.

```

Dragon Prototype:
Dragon | Health: 100 | Attack: 50 | Color: Red

Battle Dragon with Health Bonus:
Dragon | Health: 110 | Attack: 50 | Color: Red

```

Real-Time Example of Prototype Design Pattern in C#: Configuration Templates

Let's illustrate the use of the Prototype design pattern in the context of configuration templates.

- **Scenario:** A large enterprise application might have various components or modules, each requiring different configurations. However, many modules have very similar configurations or share a common base. Instead of manually creating configurations for each module, administrators can duplicate (clone) a standard configuration template and then modify it as necessary.
- **Objective:** Efficiently create module-specific configurations by cloning a standard template and adjusting the required settings.

Let us see how we can implement the above example using the Prototype Design Pattern in C#:

```

using System;
namespace PrototypeDesignPattern
{
    //Prototype - IConfigurationPrototype Interface
    public interface IConfigurationPrototype

```

```

{
    IConfigurationPrototype Clone();
}
//Concrete Prototype - Configuration Class
public class Configuration : IConfigurationPrototype
{
    public string DatabaseConnectionString { get; set; }
    public int Timeout { get; set; }
    public bool EnableLogging { get; set; }
    public string ModuleName { get; set; }
    public IConfigurationPrototype Clone()
    {
        return new Configuration
        {
            DatabaseConnectionString = this.DatabaseConnectionString,
            Timeout = this.Timeout,
            EnableLogging = this.EnableLogging,
            // ModuleName can be set separately after cloning
            ModuleName = string.Empty
        };
    }
    public void Display()
    {
        Console.WriteLine($"Configuration for {ModuleName}:");
        Console.WriteLine($"DB Connection: {DatabaseConnectionString} |
Timeout: {Timeout}s | Logging: {EnableLogging}\n");
    }
}

//Client Code
//Testing the Prototype Design Pattern
public class Program
{
    public static void Main()
    {
        // Base configuration template prepared by the IT team
        Configuration baseConfig = new Configuration
        {
            DatabaseConnectionString =
"Server=myServer;Database=myDB;User Id=myUser;Password=myPass;",
            Timeout = 30,
            EnableLogging = true,
            ModuleName = "Base Configuration"
        };
        Console.WriteLine("Base Configuration:");
        baseConfig.Display();
        // Now, for a specific module called "PaymentProcessing", we
        want a similar configuration but with a longer timeout
        Configuration paymentConfig =
(Configuration)baseConfig.Clone();
        paymentConfig.Timeout = 60;
        paymentConfig.ModuleName = "PaymentProcessing";
        Console.WriteLine("Payment Processing Module Configuration:");
        paymentConfig.Display();
    }
}

```

```

        Console.ReadKey();
    }
}

```

The IT team prepares a base configuration with common settings in this example. When setting up a specific module, instead of starting from scratch, the Prototype pattern is employed to duplicate the base configuration quickly. This cloned configuration can then be adjusted for the module-specific needs. Such an approach promotes consistency, reduces setup errors, and speeds up the configuration process. When you run the above code, you will get the following output.

```

Base Configuration:
Configuration for Base Configuration:
DB Connection: Server=myServer;Database=myDB;User Id=myUser;Password=myPass; | Timeout: 30s | Logging: True

Payment Processing Module Configuration:
Configuration for PaymentProcessing:
DB Connection: Server=myServer;Database=myDB;User Id=myUser;Password=myPass; | Timeout: 60s | Logging: True

```

Real-Time Example of Prototype Design Pattern in C#: Historical States for Undo Actions

Using the Prototype pattern to capture historical states for undo actions is a classic application of this design pattern. Let's dive into it.

- **Scenario:** Imagine you're developing a text editor application. As users make changes to the document, the application saves the state of the document after each change. When users press the "Undo" button, the editor should revert to the last saved state.
- **Objective:** Efficiently capture the state of a document after each change to facilitate the undo functionality.

Let us see how we can implement the above example using the Prototype Design Pattern in C#:

```

using System;
using System.Collections.Generic;
namespace PrototypeDesignPattern
{
    //Prototype - IDocumentStatePrototype Interface
    public interface IDocumentStatePrototype
    {
        IDocumentStatePrototype Clone();
    }
    //Concrete Prototype - DocumentState Class
    public class DocumentState : IDocumentStatePrototype
    {
        public string Content { get; set; }
        public string FontName { get; set; }
        public int FontSize { get; set; }
        public IDocumentStatePrototype Clone()
        {
            return new DocumentState
            {
                Content = this.Content,
                FontName = this.FontName,
                FontSize = this.FontSize
            };
        }
    }
}

```

```

//Client Code
//Testing the Prototype Design Pattern
public class Program
{
    private static DocumentState currentDocument = new DocumentState();
    private static Stack<DocumentState> history = new
Stack<DocumentState>();
    public static void Main()
    {
        Stack<DocumentState> history = new Stack<DocumentState>();
        // Initial document state
        DocumentState document = new DocumentState
        {
            Content = "Hello, world!",
            FontName = "Arial",
            FontSize = 12
        };
        // Save initial state
        history.Push((DocumentState)document.Clone());
        // User changes the font
        document.FontName = "Times New Roman";
        // Save state after changing font
        history.Push((DocumentState)document.Clone());
        // User adds more content
        document.Content += "\nAdding some more text.";
        // Save state after adding more content
        history.Push((DocumentState)document.Clone());
        // User decides to undo
        if (history.Count > 0)
        {
            document = history.Pop();
        }
        // Display the document's state to simulate how it would appear
in the editor
        Console.WriteLine($"Content: {document.Content}");
        Console.WriteLine($"Font: {document.FontName}");
        Console.WriteLine($"Size: {document.FontSize}");
        Console.ReadKey();
    }
}

```

In this example, every time the user changes the document, the application clones the current state of the DocumentState and pushes it onto a history stack. When the user decides to undo an operation, the application pops the last saved state from the stack and restores the document to that state.

This demonstrates the power of the Prototype design pattern in quickly saving and restoring states for undo functionality. When you run the above code, you will get the following output.

```

Content: Hello, world!
Adding some more text.
Font: Times New Roman
Size: 12

```

Advantages and Disadvantages of Prototype Design Patterns in C#

The Prototype design pattern offers a mechanism for creating new objects by copying existing ones, known as “prototypes”. Here are its advantages and disadvantages in the context of C#:

Advantages of Prototype Design Patterns in C#:

- **Performance Improvement:** The Prototype pattern can be more efficient than creating instances of an object from scratch, especially if the object creation process is resource-intensive or involves several steps.
- **Dynamic Object Creation:** Objects can be cloned at runtime without dependency on their classes. It allows for adding and removing objects at runtime.
- **Consistency with Variations:** It allows you to create a new object similar to an existing one and then make minor modifications, ensuring a consistent base object.
- **Hide Concrete Product Classes:** The pattern helps hide the concrete product classes from the client, thus adhering to the principle of least knowledge.
- **Preserve Existing Object State:** Useful for application functionalities like undo/redo. The system can capture and revert to previous states of objects.

Disadvantages of Prototype Design Patterns in C#:

- **Shallow vs. Deep Copy Issues:** The default cloning mechanism in C# provides shallow copying. If the object contains other objects or complex structures, you might need to implement deep copying, which can be complex and error-prone.
- **Overhead:** Each prototype needs to be initialized, which could be an overhead if not used wisely.
- **Maintenance:** If the object's structure changes (for example, adding new fields), you have to ensure the clone method is also updated, which could be a maintenance challenge.
- **Hidden Costs:** Sometimes, the creation process might be more than just copying, leading to hidden performance costs.
- **Not Always Clear:** The pattern's applicability isn't always clear to developers. Using it unnecessarily can lead to added complexity.

In the next article, I will discuss the [Singleton Design Pattern in C#](#) with Examples. Here, in this article, I try to explain **Real-Time Examples of the Prototype Design Patterns in C#**. I hope you enjoy this Prototype Design Pattern with Real-Time Examples using the C# article.